

به نام خدا



دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده برق و کامپیوتر



درس شبکه‌های عصبی و یادگیری عمیق

مدرس: دکتر احمد کلهر

تمرین شماره ۵

فروردین ماه ۱۴۰۵

۳	Disaster Tweet Classification with a Tiny Transformer Encoder
۳	 بخش اول: سوالات پیاده سازی (۶۵ نمره)
۴	 مقاله‌ی مرجع
۴	 دیتاست
۵	 پیش‌پردازش متن و آماده‌سازی داده (۱۲ نمره)
۶	 پیاده‌سازی Tiny Transformer Encoder از صفر (۲۶ نمره)
۸	 آموزش، ارزیابی (۱۰ نمره)
۹	 بازسازی ساده‌شده روش مقاله و مقایسه با Tiny Transformer Encoder (۱۷ نمره)
۱۱	 بخش امتیازی (۱۰ نمره اضافه)
۱۲	 بخش دوم: سوالات تئوری (۳۵ نمره)
۱۲	 سؤال ۱: مفهوم کلی Self-Attention (۴ نمره)
۱۲	 سؤال ۲: محاسبه‌ی Scaled Dot-Product Attention (۵ نمره)
۱۳	 سؤال ۳: دلیل تقسیم بر dk (۴ نمره)
۱۳	 سؤال ۴: شکل‌های Tensor در Multi-Head Attention (۵ نمره)
۱۴	 سؤال ۵: توضیح Positional Encoding (۵ نمره)
۱۵	 سؤال ۶: Encoder، Decoder و Causal Mask (۳ نمره)
۱۵	 سؤال ۷: شمارش پارامترهای یک Transformer Encoder Block (۵ نمره)
۱۶	 سؤال ۸: تحلیل Attention در مسئله‌ی توییت‌های بحران (۴ نمره)
۱۷	 سیاست استفاده از ابزارهای هوش مصنوعی و تقلب
۱۸	 نکات تحویل

DISASTER TWEET CLASSIFICATION WITH A TINY TRANSFORMER ENCODER

بخش اول: سوالات پیاده سازی (۶۵ نمره)

در سال‌های اخیر، شبکه‌های اجتماعی به یکی از مهم‌ترین منابع تولید و انتشار اطلاعات در زمان وقوع حوادث، بحران‌ها و بلایای طبیعی تبدیل شده‌اند. هنگام وقوع زلزله، سیل، آتش‌سوزی، انفجار یا سایر شرایط اضطراری، کاربران شبکه‌های اجتماعی ممکن است اطلاعات مهمی درباره‌ی محل حادثه، وضعیت افراد، نیاز به کمک، مسیرهای بسته‌شده یا هشدارهای فوری منتشر کنند.

با این حال، همه‌ی متن‌هایی که شامل کلمات مرتبط با بحران هستند، الزاماً درباره‌ی یک حادثه‌ی واقعی نیستند. برای مثال، ممکن است کاربری بنویسد:

The movie was a total disaster!

در این جمله کلمه‌ی disaster آمده است، اما جمله درباره‌ی یک حادثه‌ی واقعی نیست. از طرف دیگر، جمله‌ای مانند زیر واقعاً می‌تواند درباره‌ی یک وضعیت بحرانی باشد:

Forest fire near my town, people are being evacuated.

هدف این تمرین، طراحی و پیاده‌سازی یک مدل یادگیری عمیق برای تشخیص این موضوع است که آیا یک توییت واقعاً درباره‌ی یک حادثه یا بحران واقعی است یا نه.

در این تمرین، شما ابتدا یک مقاله‌ی کاربردی در حوزه‌ی طبقه‌بندی توییت‌های بحران را مطالعه می‌کنید. سپس با الهام از ایده‌ی مدل‌های Transformer-based، یک نسخه‌ی کوچک و آموزشی از Transformer Encoder را برای مسئله‌ی طبقه‌بندی متن پیاده‌سازی می‌کنید.

توجه کنید که هدف این تمرین fine-tune کردن مدل‌های آماده‌ی مانند BERT نیست. هدف اصلی این است که اجزای اصلی Transformer Encoder را درک کرده و بخش‌های مهم آن را خودتان پیاده‌سازی کنید.

مقاله‌ی مرجع

مقاله‌ی اصلی این تمرین Disaster Tweets Classification using BERT-Based Language Model است. این مقاله از مدل‌های مبتنی بر BERT برای تشخیص توییت‌های مرتبط با بحران استفاده می‌کند. شما باید مقاله را مطالعه کنید و ایده‌ی کلی مسئله، نقش مدل‌های Transformer-based در پردازش متن، و تفاوت مدل‌های contextual با روش‌های ساده‌تر نمایش متن را توضیح دهید.

به جهت مشکلات موجود در وضعیت اینترنت کشور، در این تمرین، لازم نیست دقیقاً مدل مقاله را بازتولید کنید. مقاله به عنوان منبع الهام و زمینه‌ی کاربردی تمرین استفاده می‌شود. پیاده‌سازی اصلی شما باید یک Tiny Transformer Encoder باشد که بدون استفاده از مدل‌های آماده‌ی Transformer نوشته می‌شود.

فایل PDF مقاله همراه منابع تمرین در اختیار شما قرار داده می‌شود.

دیتاست

در این تمرین از دیتاست **Disaster Tweets** استفاده می‌شود. هر نمونه در این دیتاست یک متن کوتاه شبیه توییت است. برچسب هر نمونه مشخص می‌کند که آیا متن واقعاً درباره‌ی یک حادثه یا بحران واقعی است یا خیر. هر نمونه شامل ستون‌های زیر است:

id : شناسه نمونه

keyword : کلمه کلیدی مرتبط با متن، در صورت وجود

location : موقعیت مکانی ثبت شده توسط کاربر، در صورت وجود

text : متن توییت

target : برچسب نهایی

برچسب‌ها به صورت زیر تعریف می‌شوند:

target = 1 → توییت درباره‌ی یک حادثه یا بحران واقعی است.

target = 0 → توییت درباره‌ی حادثه واقعی نیست.

برای جلوگیری از وابستگی به اینترنت، فایل‌های دیتاست به صورت آماده در اختیار شما قرار داده می‌شوند. شما مجاز به دانلود دیتاست از اینترنت در زمان انجام تمرین نیستید.

پیش‌پردازش متن و آماده‌سازی داده (۱۲ نمره)

در این بخش باید داده‌های خام را برای ورود به مدل آماده کنید.

فایل‌های `train.csv`، `validation.csv` و `test.csv` را بخوانید و تعداد نمونه‌های هر بخش و توزیع برچسب‌ها را گزارش کنید.

یک تابع ساده برای پاک‌سازی متن پیاده‌سازی کنید. این تابع باید متن خام هر توییت را به شکلی آماده کند که بتوان آن را به مدل داد. برای این کار، ابتدا تمام حروف متن را به `lowercase` تبدیل کنید تا کلماتی که فقط از نظر بزرگی و کوچکی حروف تفاوت دارند، به عنوان کلمات متفاوت در نظر گرفته نشوند. سپس `URL`های موجود در متن را حذف کرده یا با یک توکن مشخص مانند `<URL>` جایگزین کنید. همچنین `mention`ها مانند `user@` را حذف کرده یا با توکنی مانند `<USER>` جایگزین کنید. در ادامه، کاراکترهای غیرضروری، نشانه‌های اضافی و فاصله‌های تکراری را تا حد مناسب پاک‌سازی کنید. در نهایت، متن پاک‌سازی شده را با یک روش ساده به کلمات جداگانه تقسیم کنید تا بتوان از آن برای ساخت `vocabulary` و تبدیل متن به `sequence` عددی استفاده کرد. توجه کنید که پاک‌سازی بیش از حد متن ممکن است باعث حذف اطلاعات مفید شود؛ بنابراین تصمیم‌هایی که در این مرحله می‌گیرید باید در گزارش نهایی توضیح داده شوند. توجه کنید که پاک‌سازی بیش از حد متن ممکن است باعث حذف اطلاعات مفید شود. بنابراین تصمیم‌های خود را در گزارش توضیح دهید.

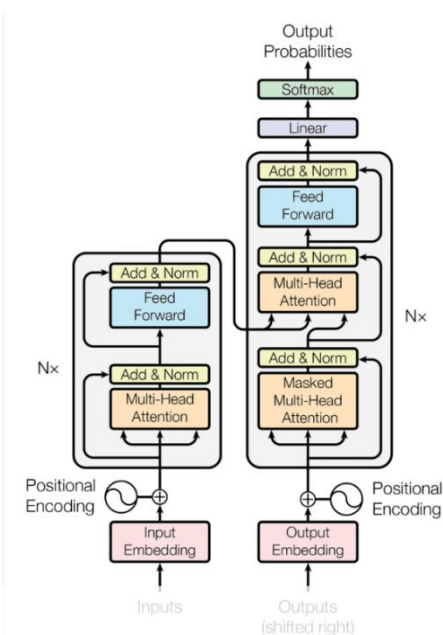
با استفاده از داده‌ی `train`، یک `vocabulary` بسازید. در این مرحله، ابتدا تمام توکن‌های موجود در متن‌های آموزشی را استخراج کرده و بر اساس آن‌ها یک نگاشت از کلمات به شناسه‌های عددی ایجاد کنید. دقت کنید که `vocabulary` فقط باید با استفاده از داده‌ی `train` ساخته شود و نباید از داده‌های `validation` یا `test` برای ساخت آن استفاده کنید. دو توکن ویژه‌ی `<PAD>` و `<UNK>` باید حتماً در `vocabulary` وجود داشته باشند. توکن `<PAD>` برای هم‌طول کردن `sequence`ها از طریق `padding` استفاده می‌شود و توکن `<UNK>` برای نمایش کلماتی به کار می‌رود که در زمان پردازش متن، در `vocabulary` دیده نشده‌اند.

هر متن پاک‌سازی شده را با استفاده از `vocabulary` ساخته شده به یک دنباله از شناسه‌های عددی تبدیل کنید؛ به این صورت که هر توکن متن با شناسه‌ی متناظر خود در `vocabulary` جایگزین شود و اگر توکنی در `vocabulary` وجود نداشت، شناسه‌ی مربوط به `<UNK>` برای آن در نظر گرفته شود. از آنجا

که ورودی مدل باید طول ثابتی داشته باشد، همه‌ی `sequence` ها را به طول مشخص `max_len` تبدیل کنید؛ اگر طول یک متن کمتر از `max_len` بود، آن را با توکن `<PAD>` کامل کنید و اگر طول آن بیشتر از `max_len` بود، فقط `max_len` توکن اول را نگه دارید و بقیه را حذف کنید.

پس از تبدیل متن‌ها به `sequence` های عددی، برای داده‌های `train`، `validation` و `test` یک کلاس `Dataset` مناسب پیاده‌سازی کنید که در هر بار فراخوانی، `sequence` عددی متن و برچسب متناظر آن را برگرداند. سپس با استفاده از این `Dataset` ها، برای هر بخش یک `DataLoader` بسازید تا داده‌ها به صورت `batch` به مدل داده شوند. دقت کنید که داده‌های آموزشی بهتر است در هر `epoch` به صورت تصادفی `shuffle` شوند، اما برای داده‌های `validation` و `test` نیازی به `shuffle` کردن وجود ندارد.

پیاده‌سازی TINY TRANSFORMER ENCODER از صفر (۲۶ نمره)



(۱) **Token Embedding**: در این بخش ابتدا باید یک لایه‌ی

`Token Embedding` پیاده‌سازی کنید. ورودی مدل در ابتدا

به صورت شناسه‌های عددی توکن‌ها است؛ یعنی هر متن پس از

مرحله‌ی پیش‌پردازش و تبدیل به `sequence`، به شکل یک

`Tensor` با ابعاد $(batch_size, sequence_length)$ وارد

مدل می‌شود. از آنجا که شناسه‌های عددی به تنهایی معنای قابل

یادگیری مناسبی برای مدل ندارند، باید هر شناسه‌ی توکن به

یک بردار پیوسته با طول `embedding_dim` تبدیل شود.

بنابراین خروجی این بخش باید `Tensor` ای با ابعاد

$(batch_size, sequence_length, embedding_dim)$

باشد. این `embedding` ها نمایش اولیه‌ی کلمات یا توکن‌ها

هستند و در ادامه به عنوان ورودی اصلی `Transformer`

`Encoder` استفاده می‌شوند.

تصویر ۱ شمای کلی مدل ترنسفورمر

(۲) **Positional Encoding**: از آنجایی که مکانیزم `Self-Attention` به صورت ذاتی ترتیب توکن‌ها

را در نظر نمی‌گیرد، لازم است اطلاعات مربوط به موقعیت هر توکن در جمله به `embedding` ها اضافه

شود. برای این کار باید `Positional Encoding` سینوسی را پیاده‌سازی کنید. مقدار `positional`

`encoding` برای موقعیت `pos` و بعد `i` با استفاده از روابط زیر محاسبه می‌شود:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

پس از محاسبه‌ی این بردارها، باید positional encoding را به token embedding اضافه کنید تا مدل علاوه بر معنای تقریبی توکن‌ها، اطلاعاتی درباره‌ی جایگاه آن‌ها در متن نیز دریافت کند.

۳ Scaled Dot-Product Self-Attention: در ادامه باید تابع Scaled Dot-Product Self-Attention را پیاده‌سازی کنید. در این مکانیزم، از روی ورودی مدل سه نمایش مختلف با نام‌های Query، Key و Value ساخته می‌شود. سپس با ضرب Query در ترانهاده‌ی Key، میزان ارتباط هر توکن با سایر توکن‌ها محاسبه می‌شود. این مقدار باید بر $\sqrt{d_k}$ تقسیم شود تا مقادیر score بیش از حد بزرگ نشوند و خروجی softmax پایدارتر باقی بماند. سپس روی scoreها تابع softmax اعمال می‌شود تا وزن‌های attention به دست آیند و در نهایت این وزن‌ها در Value ضرب می‌شوند تا خروجی attention تولید شود. پیاده‌سازی شما باید از mask نیز پشتیبانی کند تا مدل به توکن‌های <PAD> توجه نکند.

۴ Multi-Head Self-Attention: در این بخش باید Multi-Head Self-Attention را بدون استفاده از torch.nn.MultiheadAttention پیاده‌سازی کنید. ایده‌ی اصلی Multi-Head Attention این است که به جای محاسبه‌ی attention فقط در یک فضای برداری، embedding_dim به چند بخش کوچک‌تر تقسیم شود و attention به صورت موازی در چند head مختلف محاسبه شود. برای این کار ابتدا ورودی را به Q، K و V تبدیل کنید، سپس بعد embedding را بر اساس تعداد headها تقسیم کرده و attention را برای هر head جداگانه اجرا کنید. بعد از محاسبه‌ی خروجی همه‌ی headها، خروجی‌ها را concat کرده و یک linear projection نهایی روی آن‌ها اعمال کنید. در گزارش خود شکل Tensorها را در مراحل مختلف بنویسید. برای مثال، اگر batch_size = 32، sequence_length = 40، embedding_dim = 64 و num_heads = 4 باشد، باید توضیح دهید شکل Q، K، V بعد از تقسیم به headها، شکل scoreهای attention و شکل خروجی نهایی چگونه خواهد بود.

۵ Feed Forward Network: پس از بخش attention، باید یک شبکه‌ی Feed Forward کوچک برای استفاده در داخل Transformer Encoder Block پیاده‌سازی کنید. این شبکه روی

خروجی هر موقعیت از sequence به صورت مستقل اعمال می‌شود و هدف آن افزایش ظرفیت غیرخطی مدل است. توجه کنید که خروجی این شبکه باید دوباره بعد d_model داشته باشد تا بتوان آن را در ادامه با residual connection و سایر بخش‌های Encoder Block ترکیب کرد.

۶ Encoder Block: در این بخش باید یک Transformer Encoder Block کامل بسازید. این block باید شامل Multi-Head Self-Attention، residual connection، layer normalization، feed forward network، یک residual connection دیگر و یک layer normalization دیگر باشد. ساختار کلی می‌تواند به این صورت باشد که ابتدا خروجی Multi-Head Self-Attention به ورودی اصلی اضافه شود و سپس LayerNorm روی آن اعمال شود؛ بعد خروجی Feed Forward Network نیز به ورودی همان بخش اضافه شده و دوباره LayerNorm اعمال شود. همچنین می‌توانید از فرم Pre-LN استفاده کنید، اما در این صورت باید در گزارش خود توضیح دهید که از کدام ساختار استفاده کرده‌اید و تفاوت کلی آن با فرم دیگر چیست.

۷ Tiny Transformer Encoder Classifier: در پایان این بخش باید یک مدل کامل برای طبقه‌بندی متن بسازید که از اجزای پیاده‌سازی شده در قسمت‌های قبلی استفاده کند. ساختار کلی مدل باید به این صورت باشد که ابتدا شناسه‌های عددی توکن‌ها وارد مدل شوند، سپس از Token Embedding عبور کنند، بعد Positional Encoding به آن‌ها اضافه شود، سپس خروجی وارد یک یا چند Transformer Encoder Block شود و در نهایت با استفاده از یک روش pooling به یک نمایش برداری برای کل متن تبدیل شود. پس از آن، این نمایش برداری وارد یک MLP Classifier می‌شود و خروجی نهایی مدل به صورت logits برای دو کلاس تولید می‌شود. برای pooling می‌توانید از mean pooling روی توکن‌های غیر <PAD>، استفاده از اولین توکن، یا اضافه کردن یک توکن ویژه <CLS> استفاده کنید. اگر از mean pooling استفاده می‌کنید، باید دقت کنید که توکن‌های <PAD> در میانگین‌گیری لحاظ نشوند، چون این توکن‌ها بخشی از متن واقعی نیستند و می‌توانند نمایش نهایی جمله را خراب کنند.

آموزش، ارزیابی (۱۰ نمره)

در این بخش باید مدل Tiny Transformer Encoder خود را با استفاده از داده‌های train آموزش دهید. در هر epoch، مقدار loss و accuracy را هم برای داده‌های آموزشی و هم برای داده‌های

validation محاسبه و گزارش کنید تا روند یادگیری مدل قابل بررسی باشد. برای این مسئله که یک مسئله‌ی طبقه‌بندی دودسته‌ای است، می‌توانید از تابع هزینه‌ی CrossEntropyLoss استفاده کنید. همچنین پیشنهاد می‌شود برای بهینه‌سازی پارامترهای مدل از Adam استفاده کنید. در گزارش نهایی باید مقدار training loss، validation loss، training accuracy و validation accuracy را برای همه‌ی epochها ثبت و نمودارهای مربوط به روند یادگیری را رسم کنید به طور مشخص، یک نمودار برای مقایسه‌ی train loss و validation loss بر حسب epoch و یک نمودار دیگر برای مقایسه‌ی train accuracy و validation accuracy بر حسب epoch رسم کنید. توضیح دهید که آیا مدل به‌درستی در حال یادگیری بوده است یا نشانه‌هایی از underfitting یا overfitting مشاهده می‌شود. برای ارزیابی نهایی، معیارهای accuracy، precision، recall و F1-score را گزارش کنید و همچنین confusion matrix را محاسبه و در گزارش قرار دهید.

بازسازی ساده‌شده روش مقاله و مقایسه با TINY TRANSFORMER ENCODER (۱۷ نمره)

در این بخش، هدف این است که علاوه بر پیاده‌سازی Tiny Transformer Encoder از صفر، یک نسخه‌ی ساده‌شده از روش استفاده‌شده در مقاله را نیز پیاده‌سازی کنید و عملکرد آن را با مدل خودتان مقایسه کنید. مقاله‌ی مرجع از یک مدل BERT-based برای طبقه‌بندی توییت‌های بحران استفاده می‌کند. بنابراین در این بخش باید ایده‌ی اصلی مقاله را بازسازی کنید؛ یعنی از یک مدل زبانی Transformer-based از پیش‌آموزش‌دیده‌شده برای استخراج نمایش متنی استفاده کرده و سپس یک لایه‌ی طبقه‌بندی برای تشخیص دو کلاس disaster و non-disaster روی آن قرار دهید.

برای انجام این بخش، ابتدا باید متن‌ها را مطابق ورودی موردنیاز مدل BERT-based آماده کنید. برخلاف مدل Tiny Transformer که در آن خودتان vocabulary ساده می‌سازید، در این بخش باید از tokenizer مربوط به مدل از پیش‌آموزش‌دیده‌شده استفاده کنید. این tokenizer متن هر توییت را به tokenهای مناسب مدل تبدیل کرده و خروجی‌هایی مانند input_ids و attention_mask تولید می‌کند. مقدار input_ids شناسه‌ی عددی توکن‌ها را مشخص می‌کند و attention_mask نشان می‌دهد کدام موقعیت‌ها مربوط به توکن واقعی و کدام موقعیت‌ها مربوط به padding هستند.

در مرحله‌ی بعد، باید یک مدل BERT-based ساده برای طبقه‌بندی بسازید. خروجی مدل BERT-based معمولاً شامل نمایش نهایی متن است که می‌توان از نمایش توکن [CLS] یا خروجی pool شده‌ی مدل برای طبقه‌بندی استفاده کرد. سپس این نمایش را به یک لایه‌ی خطی یا یک MLP کوچک بدهید تا logits مربوط به دو کلاس تولید شود. در این بخش، استفاده از مدل از پیش‌آموزش‌دیده و کتابخانه‌ی مربوط به آن فقط برای بازسازی روش مقاله مجاز است؛

برای اینکه اجرای این بخش زمان زیادی نگیرد، می‌توانید یکی از دو روش زیر را انتخاب کنید. در روش اول، کل مدل BERT-based را fine-tune کنید، اما تعداد epochها، batch size و طول ورودی را کوچک نگه دارید. در روش دوم، پارامترهای اصلی مدل از پیش‌آموزش‌دیده‌شده را freeze کنید و فقط لایه‌ی طبقه‌بندی نهایی را آموزش دهید. اگر از روش freeze کردن استفاده می‌کنید، باید در گزارش توضیح دهید که این کار چه اثری روی سرعت آموزش، تعداد پارامترهای قابل آموزش و عملکرد نهایی دارد. همچنین باید مشخص کنید که دقیقاً کدام بخش‌های مدل آموزش داده شده‌اند و کدام بخش‌ها ثابت نگه داشته شده‌اند.

پس از آموزش مدل BERT-based، باید آن را دقیقاً مانند Tiny Transformer Encoder روی validation و test ارزیابی کنید. معیارهای accuracy، precision، recall، F1-score و confusion matrix را گزارش دهید. سپس نتایج را در یک جدول مقایسه‌ای قرار دهید تا مشخص شود کدام مدل عملکرد بهتری داشته است.

در گزارش نهایی، علاوه بر معیارهای عددی، باید تفاوت دو روش را از نظر معماری و فرآیند آموزش نیز تحلیل کنید. توضیح دهید که Tiny Transformer Encoder شما چه اجزایی را خودش یاد می‌گیرد و چه محدودیت‌هایی دارد.

توضیح دهید که مدل BERT-based چه مزیت‌هایی دارد و چرا استفاده از آن در مقاله منطقی بوده است. در جای شما، اگر مدل BERT-based بهتر بود، دلیل آن را تحلیل کنید. اگر Tiny Transformer Encoder عملکرد نزدیکی داشت، توضیح دهید چرا ممکن است در این دیتاست خاص، یک مدل کوچک هم بتواند نتیجه‌ی قابل قبولی بگیرد.

در نهایت، باید حداقل چند نمونه از پیش‌بینی‌های هر دو مدل را با هم مقایسه کنید. برای چند توییت از test set، برچسب واقعی، پیش‌بینی Tiny Transformer Encoder و پیش‌بینی مدل BERT-based را گزارش دهید. نمونه‌هایی را انتخاب کنید که یکی از مدل‌ها درست و دیگری اشتباه پیش‌بینی

کرده باشد. سپس تحلیل کنید که چرا یک مدل توانسته نمونه را درست تشخیص دهد و مدل دیگر اشتباه کرده است.

بخش امتیازی (۱۰ نمره اضافه)

برای دریافت نمره امتیازی، موارد زیر را انجام دهید:

مدل را یک بار با positional encoding و یک بار بدون positional encoding آموزش دهید. نتایج را مقایسه کنید و توضیح دهید حذف positional encoding چه اثری روی عملکرد مدل داشته است.

مدل از پیش آماده را با تعدادهای مختلف آموزش دهید. نتایج را مقایسه کنید و توضیح دهید آیا افزایش تعداد headها همیشه باعث بهتر شدن مدل می‌شود یا نه.

برای یک یا دو نمونه، attention weights از headها را بررسی کنید. لازم نیست visualization خیلی پیچیده انجام دهید، اما باید حداقل نشان دهید که مدل به چه کلماتی بیشتر توجه کرده است. سپس توضیح دهید آیا attention به کلمات مهم و مرتبط با بحران توجه کرده یا نه.

بخش دوم: سوالات تئوری (۳۵ نمره)

در این بخش، هدف بررسی فهم مفهومی و ریاضی شما از معماری Transformer، به‌ویژه Self-Attention و Transformer Encoder است. پاسخ‌ها باید با بیان خودتان نوشته شوند. در صورت نیاز، از فرمول، شکل Tensorها و توضیح مرحله‌به‌مرحله استفاده کنید.

سؤال ۱: مفهوم کلی SELF-ATTENTION (۴ نمره)

- الف) Self-Attention در یک جمله یا متن چه کاری انجام می‌دهد؟
- ب) چرا در مدل‌های Transformer، به جای پردازش ترتیبی کلمات مانند RNN، از توجه میان همه‌ی کلمات استفاده می‌شود؟
- ج) تفاوت اصلی Self-Attention با یک لایه‌ی Fully Connected ساده چیست؟
- د) در مسئله‌ی طبقه‌بندی توییت‌های بحران، چرا ارتباط میان کلمات مختلف متن می‌تواند برای تشخیص درست مهم باشد؟

سؤال ۲: محاسبه‌ی SCALED DOT-PRODUCT ATTENTION (۵ نمره)

فرض کنید ماتریس‌های Query، Key و Value به صورت زیر داده شده‌اند:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = Q$$

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} = K$$

$$\begin{bmatrix} 0 & 2 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} = V$$

$$d_k = 2$$
 همچنین فرض کنید:

به سؤالات زیر پاسخ دهید:

الف) ماتریس score خام یعنی QK^T را محاسبه کنید.

ب) ماتریس scaled score یعنی $\frac{QK^T}{\sqrt{d_k}}$ را بنویسید.

ج) فرمول کلی attention weights را با استفاده از softmax بنویسید و مشخص کنید softmax روی کدام بُعد اعمال می‌شود.

د) توضیح دهید خروجی attention چگونه از ضرب attention weights در V به دست می‌آید.

ه) به زبان خودتان توضیح دهید چرا خروجی نهایی attention را می‌توان ترکیبی وزن‌دار از بردارهای Value دانست.

سؤال ۳: دلیل تقسیم بر $\sqrt{d_k}$ (۴ نمره)

در Scaled Dot-Product Attention از رابطه‌ی زیر استفاده می‌شود:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

به سؤالات زیر پاسخ دهید:

الف) اگر مقدار dd_k بزرگ باشد، چرا مقدار dot product میان Query و Key ممکن است بزرگ شود؟

ب) بزرگ شدن بیش از حد score ها چه اثری روی خروجی softmax دارد؟

ج) چرا این مسئله می‌تواند یادگیری مدل را سخت‌تر کند؟

د) نقش تقسیم بر $\sqrt{d_k}$ در پایدارتر کردن فرآیند یادگیری چیست؟

سؤال ۴: شکل TENSOR ها در MULTI-HEAD ATTENTION (۵ نمره)

فرض کنید ورودی یک لایه‌ی Multi-Head Self-Attention دارای شکل زیر است:

$$X \in R^{32 \times 40 \times 64}$$

که در آن:

batch_size = 32, sequence_length = 40, d_model = 64, num_heads = 4

به سؤالات زیر پاسخ دهید:

الف) مقدار بُعد هر head یعنی d_head را محاسبه کنید.

ب) شکل ماتریس‌های Q، K و V قبل از تقسیم به head ها چیست؟

ج) شکل Q، K و V بعد از تقسیم به head ها چیست؟

د) شکل attention score برای هر head چیست؟

ه) پس از محاسبه attention و concat کردن خروجی head ها، شکل خروجی نهایی چیست؟

و) اگر d_model بر تعداد head ها بخش پذیر نباشد، چه مشکلی ایجاد می شود؟

سؤال ۵: توضیح POSITIONAL ENCODING (۵ نمره)

در Transformer، برخلاف RNN، ترتیب کلمات به صورت ذاتی در مدل وجود ندارد. به همین دلیل از Positional Encoding استفاده می شود.

رابطه‌ی sinusoidal positional encoding به صورت زیر است:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

به سؤالات زیر پاسخ دهید:

الف) چرا Self-Attention به تنهایی ترتیب کلمات را درک نمی کند؟

ب) در مدل ترنسفورمر، Positional Encoding چه اطلاعاتی را به مدل اضافه می کند؟

ج) در روابط بالا، نقش pos، و d_model را توضیح دهید.

د) تفاوت Positional Encoding ثابت سینوسی با Positional Embedding قابل یادگیری چیست؟

ه) اگر Positional Encoding را حذف کنیم، مدل در مسئله‌ی طبقه‌بندی متن چه اطلاعاتی را از دست می‌دهد؟

سؤال ۶: CAUSAL MASK و DECODER, ENCODER (۳ نمره)

الف) تفاوت کلی Transformer Encoder و Transformer Decoder چیست؟

ب) در Decoder چرا از Causal Mask استفاده می‌شود؟

ج) آیا در مسئله‌ی Disaster Tweet Classification به Causal Mask نیاز داریم؟ چرا؟

د) برای طبقه‌بندی متن، چرا معمولاً استفاده از Transformer Encoder کافی است؟

سؤال ۷: شمارش پارامترهای یک TRANSFORMER ENCODER BLOCK (۵ نمره)

فرض کنید یک Transformer Encoder Block با تنظیمات زیر داریم:

$$d_model = 64, num_heads = 4, d_ff = 128$$

فرض کنید در بخش Multi-Head Attention از چهار projection خطی زیر استفاده می‌شود:

$$W_Q, W_K, W_V, W_O \in R^{64 \times 64}$$

همچنین در Feed Forward Network دو لایه‌ی خطی زیر داریم:

$$W_1 \in R^{64 \times 128}, W_2 \in R^{64 \times 128}$$

به سؤالات زیر پاسخ دهید:

الف) تعداد پارامترهای بخش Multi-Head Attention را بدون در نظر گرفتن bias محاسبه کنید.

ب) تعداد پارامترهای Feed Forward Network را بدون در نظر گرفتن bias محاسبه کنید.

- ج) اگر برای هر لایه‌ی خطی bias نیز در نظر بگیریم، تعداد کل bias ها چقدر می‌شود؟
- د) توضیح دهید چرا با افزایش d_model ، تعداد پارامترهای مدل به سرعت زیاد می‌شود.
- ه) اگر تعداد Encoder Block ها از ۲ به ۴ افزایش یابد، چه اثری روی تعداد پارامترها، زمان آموزش و احتمال overfitting دارد؟

سؤال ۸: تحلیل ATTENTION در مسئله‌ی توییت‌های بحران (۴ نمره)

فرض کنید مدل برای جمله‌ی زیر پیش‌بینی کرده است که توییت مربوط به بحران واقعی است:

Emergency crews are responding to a building fire downtown.

به سؤالات زیر پاسخ دهید:

- الف) انتظار دارید مدل بیشتر به چه کلماتی در این جمله توجه کند؟
- ب) آیا توجه زیاد به کلمه‌ی fire به تنهایی برای تشخیص بحران واقعی کافی است؟ چرا؟
- ج) چرا ترکیب کلماتی مانند emergency crews ، responding و building fire می‌تواند برای تصمیم مدل مهم باشد؟
- د) یک مثال بنویسید که در آن کلمه‌ی fire وجود داشته باشد، اما جمله الزاماً درباره‌ی یک بحران واقعی نباشد.

سیاست استفاده از ابزارهای هوش مصنوعی و تقلب

استفاده از مدل‌های زبانی و ابزارهای هوش مصنوعی برای حل مستقیم تمرین، تولید کد نهایی، تولید گزارش نهایی یا پاسخ دادن به سؤالات تئوری مجاز نیست. استفاده‌ی محدود از این ابزارها فقط در موارد زیر مجاز است:

-رفع خطاهای عمومی برنامه‌نویسی

-توضیح مفاهیم کلی

-کمک در فهم error message ها

در صورت استفاده از هر ابزار هوش مصنوعی، باید در انتهای گزارش بخشی اضافه کنید و در آن توضیح دهید که از چه ابزاری استفاده کرده‌اید؟ برای چه بخشی استفاده کرده‌اید؟ چه مقدار از خروجی را تغییر داده‌اید؟

کپی کردن کد، گزارش یا پاسخ‌های تولیدشده توسط ابزارهای هوش مصنوعی بدون فهم و بازنویسی شخصی نیز تقلب محسوب می‌شود.

موارد زیر تقلب محسوب می‌شوند:

-کپی کردن کد از دیگران

-استفاده از پیاده‌سازی آماده Transformer یا BERT یا RoBERTa در بخش پیاده‌سازی از صفر

-ارسال گزارش تولیدشده توسط مدل زبانی بدون فهم و بازنویسی

-آموزش مدل روی test set

-دستکاری نتایج گزارش شده

در صورت مشاهده‌ی استفاده‌ی نامناسب از منابع، مدل‌های آماده، ابزارهای هوش مصنوعی یا کد دیگران، مطابق قوانین درس برخورد خواهد شد.

نکات تحویل

- مهلت ارسال این تمرین تا پایان روز "دوشنبه ۱ تیر ماه" خواهد بود.
- این زمان قابل تمدید نیست و در صورت نیاز می‌توانید از گریس استفاده کنید.
- در نظر داشته باشید که حداکثر مهلت آپلود تمرین در سامانه تا ۷ روز پس مهلت تحویل است و پس از آن سامانه بسته خواهد شد.
- پیاده سازی با زبان برنامه نویسی پایتون باید باشد و کدهای شما باید قابل اجرا بوده و به همراه گزارش آپلود شوند.
- انجام این تمرین به صورت یک نفره می‌باشد.
- در صورت مشاهده هر گونه تشابه در گزارش کار یا کدهای پیاده‌سازی، این امر به منزله تقلب برای طرفین در نظر گرفته خواهد شد.
- استفاده از کدهای آماده بدون ذکر منبع و بدون تغییر به منزله تقلب خواهد بود و نمره تمرین شما صفر در نظر گرفته می‌شود.
- در صورت رعایت نکردن فرمت گزارش کار نمره گزارش به شما تعلق نخواهد گرفت.
- تحویل تمرین به صورت دستنویس قابل پذیرش نیست.
- تمامی تصاویر و جداول مورد استفاده در گزارش کار باید دارای توضیح (caption) و شماره باشند.
- بخش زیادی از نمره شما مربوط به گزارش کار و روند حل مسئله است.
- لطفا گزارش، فایل کدها و سایر ضمیمات مورد نیاز را با فرمت زیر در سامانه بارگذاری نمایید.
- HW5_[Lastname]_[StudentNumber].zip
- در صورت وجود سوال و یا ابهام می‌توانید از طریق رایانامه زیر با موضوع NNDL_HW5 با دستیاران آموزشی در ارتباط باشید:

○ دستیار تمرین: MohammadAmanlou2@gmail.com – Mohammad.Amanlou@ut.ac.ir

با آرزوی سلامتی و موفقیت روزافزون